

MongoDB WiredTiger Cache Incident — Root Cause Analysis

Periodic update-cache spikes & application-thread throttling

Host: `hbl-gcp-cxh-mongo-case-47-prod-vm-as2` | MongoDB 7.0.28 (WiredTiger), 88 cores, 346 GB RAM, ~172 GB cache
Incident window analysed: 2026-05-20 17:03 → 2026-05-21 09:20 UTC (server local +08:00) | Source: FTDC diagnostic.data

Executive summary. Every ~3 minutes the WiredTiger **update-cache ratio** spiked from a ~2.5% baseline to **10–14.5%**, crossing the `eviction_updates_trigger` (10%) and forcing MongoDB's own application threads to perform eviction (latency-impacting throttling). The **dirty** ratio never approached its 20% trigger. Root cause: each WiredTiger **checkpoint** ends with a **history-store (HS) flush**; during that phase WiredTiger **cannot evict the dirty HS pages it is writing**, so they pile up in cache and trip the update trigger, then drain when the checkpoint completes. Checkpoints take ~130 s because each writes ~150 GB at a ~1.1 GB/s storage ceiling. The underlying driver is **large, frequently-updated documents** generating a large volume of historical versions. **The durable fix is reducing document size / update churn**, not tuning the eviction thresholds.

1. What was observed

The update-cache ratio (bytes allocated for updates ÷ cache size) showed a regular sawtooth: a flat ~2.5% baseline interrupted by a sharp spike to 10–14.5% roughly every 3 minutes. Each spike crosses the 10% trigger, at which point WiredTiger conscripts application threads into eviction — the source of the latency symptoms. The dirty ratio stayed well below its 20% trigger throughout (peak ~14.7%, mostly 2–6%).

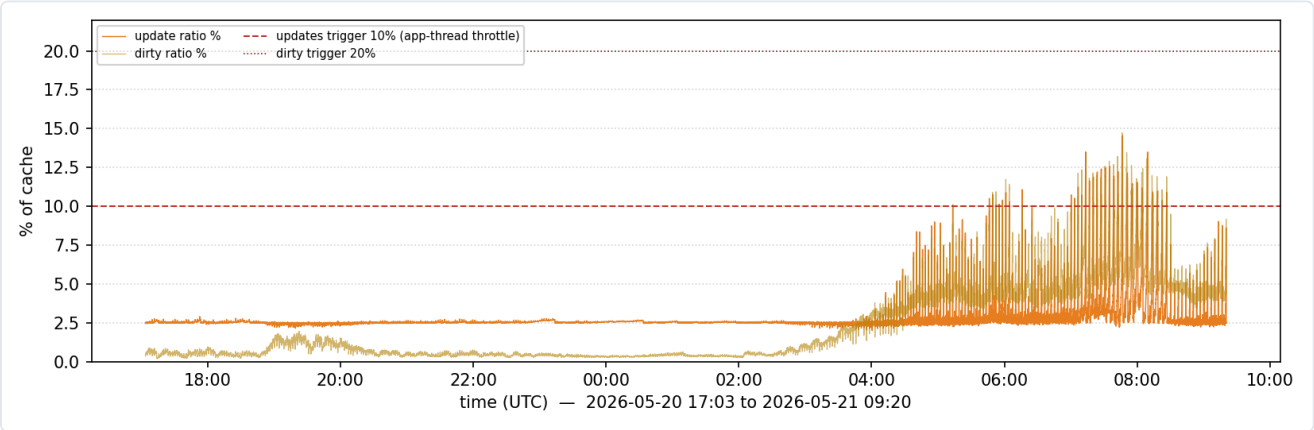


Figure 1 — Full incident window. Orange = update ratio %, amber = dirty ratio %. The update ratio repeatedly hits the 10% trigger (red dashed); the dirty ratio never reaches its 20% trigger.

Metric	Value during incident
Update ratio — baseline / peak	~2.5% / 14.54% (max)
Dirty ratio — peak	14.74% (never hit 20% trigger)

Total cache used	~80–90%
Spike period	~196 s (≈ every 3 minutes)
Spike width	~15–20 s
Samples ≥10% updates	630 (1-second samples)

2. Background: update ratio vs dirty ratio

WiredTiger limits how much of its cache may be un-written. Two of those limits matter here:

- **Update bytes** (`eviction_updates_target/trigger` , 2.5% / 10%): memory held by *update structures* — the per-record modification chains not yet reconciled into page images. **This is what spiked.**
- **Dirty bytes** (`eviction_dirty_target/trigger` , 5% / 20%): total size of all dirty pages. Update bytes are a subset of dirty bytes.

Crossing a **target** makes background eviction work harder; crossing a **trigger** forces **application threads** to do eviction before their writes proceed — i.e. client operations stall. The incident is the **updates trigger** being crossed while the dirty trigger is not.

3. Investigation & findings

3.1 The spikes track the checkpoint cycle

The ~3-minute period equals one checkpoint duration (~130 s) plus the ~60 s wait between checkpoints. Every spike occurs while a checkpoint is running and collapses the instant the checkpoint completes. There is exactly one spike per checkpoint.

3.2 The spike is the history store filling the cache

At 1-second resolution, history-store bytes in cache track the update-ratio spike **1:1**: HS in cache grows from ~300 MB to **18–27 GB** during the spike and collapses afterward. At the worst spike, HS in cache = 27.4 GB = 14.8% of cache — identical to the 14.54% update-ratio peak. **The update-ratio spike is the history store.**

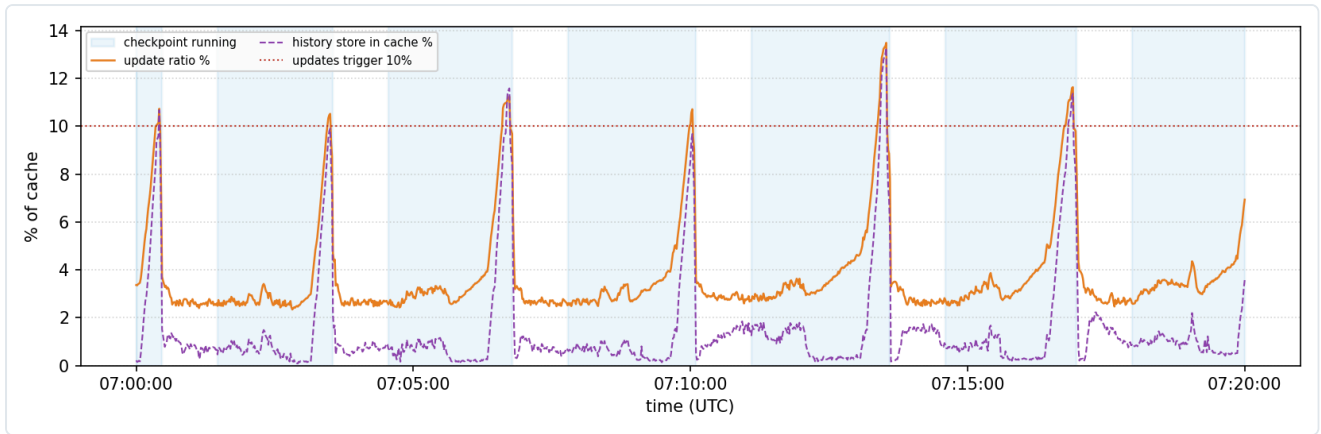


Figure 2 — A 20-minute slice. Blue bands = checkpoint running. Update ratio (orange) and history-store-in-cache (purple dashed) stay flat for most of each checkpoint, then spike together in its final seconds and collapse when the checkpoint ends.

3.3 Ruled out: write bursts, long readers, replication lag

Hypothesis	Evidence	Verdict
Periodic application write burst	<code>opcounters/update</code> ≈ 0 during spikes	Ruled out
Long-lived readers / pinned snapshots	snapshot window ~ 5 s; 0 SnapshotTooOld; ≤ 9 pinned cursors	Ruled out
Replication / majority-commit lag	member optime spread ~ 1.6 s during the spikes	Ruled out

3.4 Confirmed mechanism: eviction is blocked during the history-store checkpoint

WiredTiger checkpoints the history-store table **after** the data files (documented behaviour — see Appendix). We confirmed from FTDC that the spike coincides exactly with this phase, using the `transaction checkpoint currently running for history store file gauge`. During that phase the counter `eviction server skips dirty pages during a running checkpoint` explodes from $\sim 7,000/s$ to **75,000/s**, while successful dirty evictions stay flat — i.e. WiredTiger **cannot evict the dirty HS pages it is checkpointing**, so they accumulate. When update bytes cross 10%, `pages evicted by application threads` jumps from 0 to $\sim 47,000/s$ — the throttling. When the phase ends, eviction resumes and the cache drains.

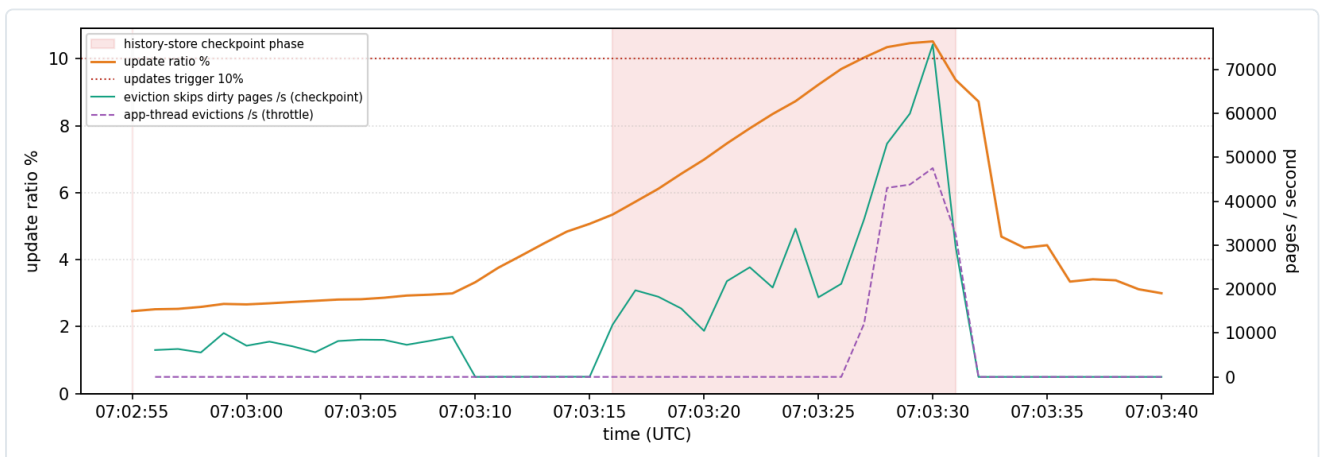


Figure 3 — One spike at 1-second resolution. Red band = history-store checkpoint phase. The update ratio (orange) balloons through it; "eviction skips dirty pages during checkpoint" (teal) spikes to $\sim 75k/s$; application-thread evictions (purple) — the throttle — start as the ratio crosses 10%. Everything collapses when the HS phase ends.

3.5 Why each checkpoint takes ~ 130 s

Each checkpoint in the spike window **writes ~130–164 GB** at a sustained **~1.0–1.16 GB/s** — the duration is bound by storage write throughput ($\approx 150 \text{ GB} \div 1.1 \text{ GB/s} \approx 130 \text{ s}$). The dirty working set is only ~10–17 GB, so the ~150 GB written represents roughly **10× write amplification**, driven by large-document version churn and the resulting history-store traffic. Checkpoint sub-phases (scrub, handle-gathering, prepare) are all negligible (~0.1 s); the time is in the bulk page-write phase plus the 15–25 s history-store-file phase.

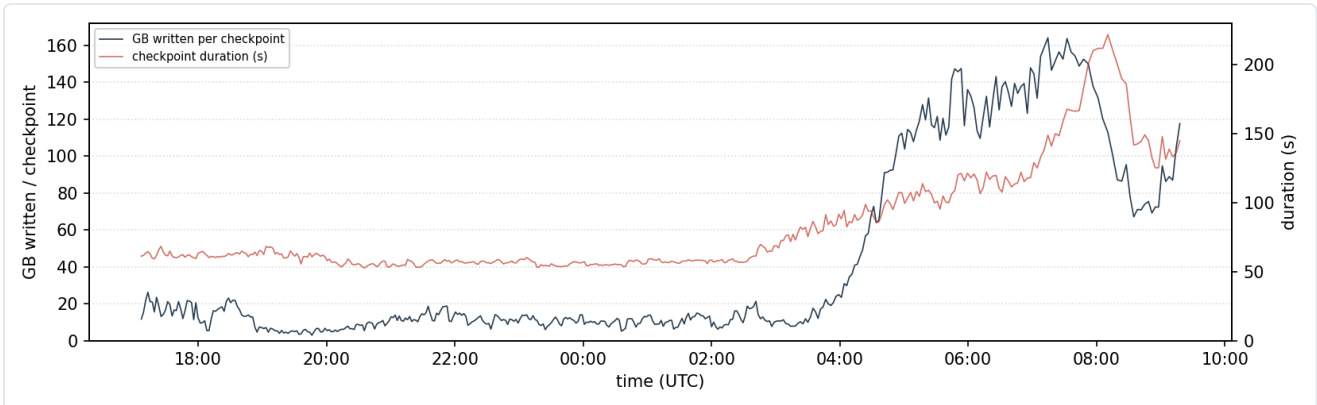


Figure 4 — Per-checkpoint bytes written (dark) and checkpoint duration (red) across the window. Duration tracks the volume written, against the ~1.1 GB/s storage ceiling.

4. Root cause

Causal chain. Large, frequently-updated documents (large payloads / unbounded arrays) → a large volume of historical versions per checkpoint interval → each checkpoint must flush ~150 GB ($\approx 10\times$ the dirty set) to disk, taking ~130 s at the ~1.1 GB/s ceiling → in the final ~15–20 s the **history-store checkpoint phase** writes the HS, during which WiredTiger **skips/cannot evict the dirty HS pages** → HS pages pile up in cache, the update ratio crosses the 10% trigger → application threads are forced into eviction (client-visible throttling) → the phase completes, pages drain, and it repeats at the next checkpoint (~every 3 min).

It is fundamentally a **write-volume** problem (too much version history flowing through the history store each checkpoint), **not** a retention problem (readers/replication are healthy) and **not** an eviction-threshold mis-tuning.

5. Recommendations

1. **Reduce document size — cap / bucket unbounded arrays (highest leverage).** Smaller documents cut both the per-update write size and the history-store volume, directly shrinking the ~150 GB-per-checkpoint that drives the duration and the HS-flush spike.
2. **Reduce repeated updates to the same hot documents** (fewer versions per checkpoint interval) — e.g. avoid long runs of consecutive updates to a single document; batch or coalesce where possible.
3. **Increase storage write throughput** if write volume cannot be reduced soon — raising the ~1.1 GB/s ceiling shortens checkpoints and the HS-flush window. This is a mitigation, not a cure.
4. **Do not rely on raising `eviction_updates_trigger`.** It would defer the application-thread throttle at each peak but does not reduce the 150 GB write or the HS burst; treat only as a temporary stop-gap if latency spikes are unacceptable in the interim.

6. Appendix — methodology & references

Data & tooling. Analysis is from the server's FTDC (`diagnostic.data`) — full `serverStatus` sampled ~1/s. Files were decoded with a custom Go decoder (built on the open-source `simagix/mongo-ftdc` library) to extract WiredTiger cache, history-store, checkpoint and eviction counters that standard tools do not surface (notably `bytes allocated for updates` and `eviction server skips dirty pages during a running checkpoint`). Figures plotted with matplotlib.

Key metrics referenced (all under `serverStatus/wiredTiger/`): `cache/bytes allocated for updates` , `cache/tracked dirty bytes in the cache` , `cache/bytes belonging to the history store table in the cache` , `cache/eviction server skips dirty pages during a running checkpoint` , `cache/pages evicted by application threads` , `transaction/transaction checkpoint currently running for history store file` , `transaction/transaction checkpoint most recent time (msecs)` .

References. WiredTiger Architecture Guide — Checkpoint (history store checkpointed after data files):

<https://source.wiredtiger.com/develop/arch-checkpoint.html> · History Store: <https://source.wiredtiger.com/develop/arch-hs.html> · Eviction: <https://source.wiredtiger.com/develop/arch-eviction.html> · Rollback to Stable: <https://source.wiredtiger.com/develop/arch-rts.html> · MongoDB WiredTiger storage engine: <https://www.mongodb.com/docs/manual/core/wiredtiger/>

Note on confidence. The checkpoint-phase ordering (history store after data files) is documented. The eviction-blocking behaviour during that phase is confirmed empirically from this server's own FTDC counters (Figure 3), not merely inferred.